

Class 4: Containerisation with Docker

Session Overview

This session will cover the fundamental concepts of containerization, focusing on Docker as the primary platform. Students will learn the benefits of using containers, how to build and manage Docker containers, and best practices in Docker usage. The session also includes a detailed comparison between traditional virtualization using Virtual Machines (VMs) and modern containerization techniques.

Prerequisites

- Basic understanding of virtualization concepts.
- Familiarity with Linux command-line operations.
- Understanding of software development and deployment processes.

Introduction

In today's fast-paced software development landscape, delivering high-quality applications quickly and reliably is crucial. As organizations embrace DevOps practices, the need for consistency across development, testing, and production environments has become more pressing. This is where containerization comes into play.

Containerization is a method of virtualizing the operating system, allowing multiple applications to share the same OS kernel while remaining isolated from one another. Unlike traditional Virtual Machines (VMs), which virtualize the hardware and require a full OS for each application, containers package only the application and its dependencies. This results in lightweight, portable, and efficient application environments that can run consistently across various platforms.

Docker, the leading platform for containerization, has revolutionized the way developers build, ship, and run applications. With Docker, applications are not just developed faster, but they are also easier to manage, scale, and deploy. Docker containers encapsulate everything an application needs—code, runtime, system tools, libraries, and settings—into a single package that can be deployed anywhere, from a developer's laptop to large-scale production environments in the cloud.

Importance of Containerization in DevOps

In the context of DevOps, where the primary goal is to shorten the software development lifecycle and provide continuous delivery with high software quality, containerization plays a pivotal role. Containers ensure that applications behave the same in development, testing, and production environments, thereby reducing the "it works on my machine" problem. This consistency accelerates development processes, enhances collaboration between teams, and improves the overall stability of software releases.

Key Benefits of Containerization in DevOps:

- **Portability:** Containers can run consistently on any platform that supports Docker, be it a developer's machine, a testing environment, or a production server.
- **Scalability:** Containers can be easily scaled up or down to meet demand, making them ideal for microservices architectures where applications are composed of many small, independently deployable services.
- **Efficiency:** Containers use less system resources compared to traditional VMs because they share the host system's OS kernel, leading to faster startup times and lower overhead.
- **Isolation:** Each container runs in its own isolated environment, ensuring that applications do not interfere with each other, which is crucial for maintaining stability and security.

Containerisation v/s Virtualisation

In modern software development and deployment environments, both containerization and virtualization are key technologies that enable the efficient use of computing resources. However, they operate at different levels of abstraction and serve distinct purposes. Understanding the differences between these two technologies is crucial for making informed decisions about infrastructure design and application deployment.

Virtualization

Virtualization is the process of creating a virtual version of something, such as an operating system (OS), server, storage device, or network resources. It allows multiple virtual machines (VMs) to run on a single physical machine, sharing its resources such as CPU, memory, and storage. Each VM operates as a separate entity with its own OS and applications, independent of other VMs on the same host.

Key Characteristics of Virtualization:

- **Hardware Emulation:** Virtualization involves emulating the hardware of a physical machine. Each VM requires its own full operating system, which runs on a virtualized layer of the host's hardware. This layer is managed by a hypervisor.
- **Hypervisor:** The hypervisor is a software layer that sits between the physical hardware and the virtual machines. It manages the VMs, allocates resources, and ensures isolation between them. There are two types of hypervisors:
 - **Type 1 (Bare-Metal):** Runs directly on the host's hardware (e.g., VMware ESXi, Microsoft Hyper-V).
 - **Type 2 (Hosted):** Runs on a host operating system as an application (e.g., VMware Workstation, Oracle VirtualBox).
- **Resource Requirements:** Since each VM needs a full OS, virtualization can be resource-intensive. This leads to higher CPU, memory, and storage usage, as well as longer boot times compared to containers.
- **Isolation:** VMs are fully isolated from each other, meaning that processes running inside one VM cannot interfere with those in another. This strong isolation makes VMs suitable for running applications that require different OS environments or have stringent security requirements.

Advantages of Virtualization:

- **Compatibility:** VMs can run different operating systems on the same physical machine, making it possible to run Windows and Linux side by side.
- **Strong Isolation:** Full OS-level isolation ensures high security and stability, which is beneficial for running legacy applications or applications with specific OS dependencies.
- **Mature Technology:** Virtualization has been around for a long time and is a well-established technology with broad support and robust management tools.

Containerization

Containerization is a lightweight alternative to virtualization that allows applications to be packaged with all their dependencies into a container. Containers share the host system's OS kernel but are isolated from each other at the process level. This makes containers much more lightweight and efficient compared to VMs.

Key Characteristics of Containerization:

- **OS-Level Virtualization:** Containers virtualize the operating system rather than the hardware. They share the same OS kernel but maintain isolated user spaces, making them more resource-efficient.
- **Container Engine:** A container engine, like Docker, manages the creation, deployment, and management of containers. The engine ensures that containers are isolated and that resources are appropriately allocated.
- **Docker Images and Containers:** A Docker image is a read-only template that defines what is inside a container (application code, runtime, libraries). Containers are running instances of Docker images, encapsulating everything the application needs to run.
- **Efficiency and Speed:** Since containers share the host OS kernel and do not require a full OS for each instance, they are much faster to start and use fewer resources. This leads to quicker deployments and better scalability.
- **Isolation:** Containers provide process-level isolation, which is sufficient for most use cases. However, since they share the host OS, they are not as isolated as VMs. Tools like Docker ensure that containers do not interfere with each other, but they may still share the same vulnerabilities if the host OS is compromised.

Advantages of Containerization:

- **Portability:** Containers are highly portable across different environments (development, testing, production) because they encapsulate all dependencies, ensuring that applications run consistently everywhere.
- **Scalability:** Containers are lightweight and can be quickly scaled up or down to meet demand, making them ideal for microservices and cloud-native applications.
- **Resource Efficiency:** Containers use fewer resources than VMs because they share the host OS kernel, leading to lower overhead and faster startup times.
- **Rapid Deployment:** The lightweight nature of containers allows for faster deployment and continuous delivery, enabling quicker development cycles.

Feature	Virtual Machines (VMs)	Containers
Isolation Level	Hardware-level	OS-level
Resource Usage	High (full OS per VM)	Low (shared OS kernel)
Startup Time	Slow	Fast
Portability	Limited (requires hypervisor)	High (runs anywhere Docker is supported)
Overhead	High (due to full OS)	Minimal (only application dependencies)
Scalability	Moderate	High (due to low resource consumption)
Security	Strong OS-level isolation	Process-level isolation
Deployment Speed	Slow (full OS boot required)	Fast (lightweight and minimal overhead)
Use Cases	Legacy applications, multiple OS	Microservices, cloud-native applications

Docker

Docker is a powerful platform that enables developers and IT operations teams to build, deploy, and manage applications in a consistent and efficient manner. It is based on the concept of containerization, which allows applications to be packaged with all their dependencies into lightweight, portable containers that can run consistently across different environments. Docker has revolutionized the way software is developed and deployed, making it a cornerstone of modern DevOps practices.

What is Docker?

Docker is an open-source platform that automates the deployment of applications inside containers. It simplifies the process of packaging an application along with its dependencies (such as libraries, configuration files, and runtime environments) into a single, self-contained unit called a container. These containers can run on any machine that has Docker installed, ensuring that the application behaves the same way regardless of where it is deployed.

Key Components of Docker:

- Docker Images:**
 - A Docker image is a lightweight, stand-alone, and executable package that includes everything needed to run a piece of software: the code, a runtime, libraries, environment variables, and configuration files. Docker images are read-only templates from which Docker containers are instantiated.
- Docker Containers:**
 - A Docker container is a running instance of a Docker image. It is a standardized unit of software that packages up code and all its dependencies so that the application runs quickly and reliably across different computing environments. Containers are isolated from each other and the host system, but they can interact with each other as needed.
- Dockerfile:**
 - A Dockerfile is a text document that contains a series of commands used to assemble a Docker image. It automates the creation of Docker images by specifying the

base image to use, the application code to add, the dependencies to install, and the commands to run to configure the environment.

4. **Docker Engine:**

- The Docker Engine is the core of Docker. It is a lightweight runtime that allows developers to build and run containers on a host machine. The Docker Engine can run on various operating systems, including Linux, Windows, and macOS.

5. **Docker Hub:**

- Docker Hub is a cloud-based registry service where Docker users and partners create, test, store, and distribute Docker images. It is the world's largest library and community for container images, offering a vast collection of trusted and official images for various applications and services.

Why Use Docker?

Docker has become a fundamental tool in the software development lifecycle due to its ability to standardize and streamline the deployment process. Here are some key reasons why Docker is widely adopted:

1. **Portability:**

- Docker containers can run on any system that supports Docker, whether it's a developer's local machine, an on-premise server, or a cloud environment. This portability ensures that applications behave consistently across different environments, reducing the "it works on my machine" problem.

2. **Efficiency:**

- Unlike traditional virtualization, Docker containers share the host operating system's kernel, making them more lightweight and efficient. Containers start quickly and consume fewer resources than virtual machines, allowing for better resource utilization.

3. **Isolation:**

- Docker containers provide process-level isolation, ensuring that applications and their dependencies run in a secure and isolated environment. This isolation reduces conflicts between different applications running on the same host.

4. **Scalability:**

- Docker makes it easy to scale applications horizontally by running multiple containers of the same application. This is particularly useful in microservices architectures, where each service can be containerized and scaled independently.

5. **Continuous Integration and Continuous Deployment (CI/CD):**

- Docker is a critical enabler of CI/CD pipelines. By containerizing applications, teams can automate the build, test, and deployment processes, leading to faster development cycles and more reliable releases.

6. **Simplified Dependency Management:**

- Docker encapsulates all the dependencies an application needs, ensuring that it runs consistently regardless of the underlying infrastructure. This makes it easier to manage complex dependencies and avoid version conflicts.

Docker's Role in DevOps

Docker plays a crucial role in the DevOps lifecycle by bridging the gap between development and operations. It allows developers to build, test, and deploy applications in isolated containers that

work consistently across different environments. This consistency reduces the friction between development and operations teams, enabling smoother collaboration and faster delivery of software. Docker also supports modern development practices like microservices, where applications are broken down into smaller, independent services. Each microservice can be containerized, allowing for more granular control over deployment, scaling, and management.

Docker Architecture

Docker architecture is designed to facilitate the creation, deployment, and management of containers, making it an essential tool in the DevOps lifecycle. Docker provides a lightweight and efficient environment for running applications, leveraging the power of containers. The architecture of Docker consists of several key components that work together to build, ship, and run containers.

Key Components of Docker Architecture

1. **Docker Client**
 - The Docker Client is the primary interface that users interact with. It allows users to communicate with the Docker Daemon through the use of Docker commands. The client can be a command-line interface (CLI) or a graphical user interface (GUI). When you type a Docker command (e.g., `docker build`, `docker run`), the client sends these commands to the Docker Daemon, which carries out the instructions.
2. **Docker Daemon (Docker Engine)**
 - The Docker Daemon, also known as the Docker Engine, is the core component of Docker. It runs on the host machine and is responsible for building, running, and managing Docker containers. The daemon listens for Docker API requests and processes them. It is also responsible for managing Docker objects such as images, containers, networks, and volumes.
3. **Docker Images**
 - Docker Images are read-only templates used to create containers. An image includes everything needed to run an application, including the code, runtime, libraries, environment variables, and configuration files. Images are created using Dockerfiles, which contain a series of instructions on how to build the image. Once built, images are stored in a registry and can be pulled to create containers.
4. **Docker Containers**
 - Docker Containers are the runtime instances of Docker images. They are lightweight, isolated environments that run the applications. Containers share the host system's kernel but are otherwise isolated from each other and the host system. This isolation ensures that the application runs consistently, regardless of the environment in which the container is deployed.
5. **Docker Registries**
 - Docker Registries are storage and distribution systems for Docker images. Docker Hub is the default public registry, where users can store and share images. Private registries can also be set up within an organization to store proprietary images. When you use the `docker pull` or `docker push` commands, you are interacting with a Docker registry to download or upload images.

6. **Dockerfile**
 - A Dockerfile is a script containing a series of instructions on how to build a Docker image. It defines the base image, application code, dependencies, environment variables, and any other configurations needed to create the image. Dockerfiles are crucial for automating the process of creating Docker images.
7. **Docker Networking**
 - Docker Networking facilitates communication between Docker containers, whether on the same host or across different hosts. Docker provides several network drivers such as bridge, host, and overlay, each serving different networking needs. For example, the bridge network allows containers on the same host to communicate, while the overlay network enables multi-host communication.
8. **Docker Volumes**
 - Docker Volumes are used to persist data generated by containers. Unlike the container's filesystem, which is ephemeral, volumes provide a way to store data that can be shared across multiple containers or persisted beyond the container's lifecycle. Volumes can be managed using Docker commands and are the preferred way to manage data in Dockerized applications.

How Docker Architecture Works

1. **Build Phase**
 - In the build phase, a Dockerfile is used to create a Docker image. The Docker Client sends a build command to the Docker Daemon, which reads the Dockerfile and executes the instructions to create an image. Once the image is built, it is stored in a registry (either Docker Hub or a private registry).
2. **Ship Phase**
 - In the ship phase, the Docker image is distributed from the registry to various environments (e.g., development, testing, production). The image can be pulled from the registry by any Docker-enabled host machine, ensuring consistency across different environments.
3. **Run Phase**
 - In the run phase, the Docker image is instantiated into one or more containers on the target host. The Docker Daemon on the host machine starts the container(s) based on the image. The application runs inside the container, isolated from other containers and the host system. Docker Networking and Volumes can be configured to manage communication and data storage for the running containers.

Docker Architecture in Practice

- **Development:** Developers can use Docker to create consistent development environments. By building Docker images with all necessary dependencies, they ensure that the application runs the same way on their local machines as it does in production.
- **Testing:** Docker containers provide an isolated environment for testing applications. CI/CD pipelines can spin up containers to test code in environments identical to production, catching bugs early in the development process.
- **Deployment:** Docker containers make it easy to deploy applications across different environments. Whether deploying to on-premise servers, cloud platforms, or hybrid environments, Docker ensures that the application behaves consistently.

- **Scalability:** Docker allows for easy scaling of applications. By running multiple instances of a containerized application, organizations can handle increased load and ensure high availability.

Docker file

A Dockerfile is a script consisting of a series of instructions on how to build a Docker image. It's essentially a blueprint that tells Docker how to construct an image layer by layer. Dockerfiles are written in a simple, straightforward syntax that anyone familiar with shell scripting can quickly pick up.

Key Concepts and Syntax

1. FROM

- The FROM instruction is always the first line of a Dockerfile. It specifies the base image to be used for creating the new image. This could be any official image from Docker Hub or a custom image.
- Example:

```
FROM node:14
```

Here, we're using the official Node.js image as the base image.

2. WORKDIR

- The WORKDIR instruction sets the working directory inside the container. If the directory doesn't exist, it will be created.
- Example:

```
WORKDIR /app
```

This sets /app as the working directory inside the container.

3. COPY

- The COPY instruction copies files and directories from the host machine to the Docker container.
- Example:

```
COPY . .
```

This command copies all files from the current directory on the host machine to the working directory in the container.

4. RUN

- The RUN instruction allows you to execute commands inside the Docker container during the image-building process. It's often used for installing dependencies.

- Example:

```
RUN npm install
```

This installs the Node.js dependencies defined in the package.json file.

5. **CMD**

- The CMD instruction specifies the command that will be run when the container starts. Unlike RUN, which is executed during the image build, CMD is executed when a container starts.
- Example:

```
CMD ["npm", "start"]
```

This command starts the Node.js application inside the container.

6. **EXPOSE**

- The EXPOSE instruction tells Docker which port the container will listen on at runtime. This is mostly informational; to actually expose the port, you'll use the -p flag when running the container.
- Example:

```
EXPOSE 3000
```

This exposes port 3000 on the container.

7. **ENV**

- The ENV instruction sets environment variables inside the container.
- Example:

```
ENV NODE_ENV=production
```

This sets the environment variable NODE_ENV to production.

8. **ENTRYPOINT**

- The ENTRYPOINT instruction allows you to configure a container to run as an executable.
- Example:

```
ENTRYPOINT ["docker-entrypoint.sh"]
```

Example Dockerfile

Here's a complete example of a Dockerfile for a simple Node.js application:

```
# Use the official Node.js image as a base
FROM node:14

# Set the working directory inside the container
WORKDIR /app

# Copy the package.json and package-lock.json files
COPY package*.json ./

# Install the dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the application's port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]
```

This Dockerfile does the following:

- It starts with an official Node.js base image.
- It sets the working directory to /app.
- It copies the necessary files and installs dependencies.
- It exposes port 3000 and runs the application using npm start.

Best Practices for Writing Dockerfiles

1. **Use a Minimal Base Image**
 - Always start with the smallest possible base image to minimize the attack surface and improve build times.
2. **Leverage Caching**
 - Docker caches each layer during the build process. Structuring your Dockerfile to take advantage of caching can significantly speed up builds.
3. **Combine Commands**
 - Combine commands using && to reduce the number of layers in your image.
 - Example:

```
RUN apt-get update && apt-get install -y curl
```

4. **Minimize the Number of Layers**

- Each instruction in a Dockerfile creates a new layer. Combine steps where possible to reduce the final image size.
5. **Keep the Dockerfile Clean**
 - Remove unnecessary files and dependencies at the end of the Dockerfile to keep the image as small as possible.

Hands on lab

1. Install Docker

1. Go to the website <https://docs.docker.com/docker-for-windows/install/> and download the docker file.

Note: A 64-bit processor and 4GB system RAM are the hardware prerequisites required to successfully run Docker on Windows 10.

2. Then, double-click on the Docker Desktop Installer.exe to run the installer.

Note: Suppose the installer (Docker Desktop Installer.exe) is not downloaded; you can get it from Docker Hub and run it whenever required.

3. Once you start the installation process, always enable Hyper-V Windows Feature on the Configuration page.

4. Then, follow the installation process to allow the installer and wait till the process is done.

5. After completion of the installation process, click Close and restart.

2. Verify Docker Installation

1. Check Docker Version:

- Open Command Prompt or PowerShell and run:

```
docker --version
```

2. Check Docker Status:

- Run the following command to verify that Docker is running:

```
docker info
```

3. Create a Dockerfile

1. Set Up Your Project Directory:

- Open Command Prompt or PowerShell and create a new directory for your project:

```
mkdir my-docker-project
cd my-docker-project
```

2. Create a Simple Node.js Application:

- Create a file named app.js in the my-docker-project directory with the following content:

```
const http = require('http');
const port = 3000;

const requestHandler = (request, response) => {
  response.end('Hello Docker!');
};

const server = http.createServer(requestHandler);

server.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

- Create a package.json file to manage dependencies:

```
{
  "name": "my-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {}
}
```

3. Create a Dockerfile:

- In the same directory, create a file named Dockerfile with the following content:

```
# Use the official Node.js image as a base
FROM node:14

# Set the working directory inside the container
WORKDIR /app

# Copy package.json and package-lock.json
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the application's port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]
```

4. Build a Docker Image

1. Build the Docker Image:

- Run the following command to build the Docker image from the Dockerfile:

```
docker build -t my-docker-app .
```

- -t my-docker-app tags the image with the name my-docker-app.

5. Run the Docker Container

1. Run the Container:

- Start a container from the image you just built:

```
docker run -p 3000:3000 my-docker-app
```

- This maps port 3000 on your host to port 3000 in the container.
- ### 2. Verify the Application:
- Open a browser and navigate to <http://localhost:3000>. You should see the text “Hello Docker!”.

6. Manage Docker Containers

1. List Running Containers:

- To see all running containers:

```
docker ps
```

2. Stop a Container:

- Find the container ID or name from the docker ps output and stop it:

```
docker stop <container_id>
```

3. Remove a Container:

- To remove a stopped container:

```
docker rm <container_id>
```

7. Push and Pull Docker Images

1. Log In to Docker Hub:

- Log in to Docker Hub:

```
docker login
```

2. Tag Your Image:

- Tag the image for Docker Hub:

```
docker tag my-docker-app <your_dockerhub_username>/my-docker-app
```

3. **Push the Image:**

- Push the image to Docker Hub:

```
docker push <your_dockerhub_username>/my-docker-app
```

4. **Pull the Image:**

- Pull the image from Docker Hub:

```
docker pull <your_dockerhub_username>/my-docker-app
```

5. **Committing containers:**

- Committing running containers into image

```
docker commit -a "Your Name" -m "Updated configuration" abc123 myapp:v1
```

8. Docker Best Practices

1. **Minimize Image Size:**

- Use minimal base images (e.g., alpine versions).
- Remove unnecessary files and dependencies.

2. **Optimize Dockerfile Layers:**

- Combine RUN commands to reduce the number of layers:

```
RUN apt-get update && apt-get install -y curl
```

3. **Use .dockerignore:**

- Create a .dockerignore file to exclude unnecessary files from the build context:

```
node_modules
*.log
```

4. **Keep Your Dockerfile Clean:**

- Maintain a clear and organized Dockerfile for easier management and readability.



codingninjas